Pattern Matching for Java Objects 1



Chapter Topics

- Using *type patterns* with the `instanceof` *pattern match operator* and the *enhanced* `switch` *construct* to *type check objects*.
- Using *record patterns* with the `instanceof` *pattern match operator* and the *enhanced* `switch` *construct* to *deconstruct records*.



Prerequisites

- *Type checking* objects using the `instanceof` *type comparison operator*.
- Writing multi-way branching code using the *traditional* `switch` *statement* and the *traditional* `switch` *expression* that allow *constant expressions* as `case` labels (collectively called the *traditional* `switch` *construct* as opposed to the *enhanced* `switch` construct that allows pattern matching).

Java SE 21 Developer Professional Exam Objectives	
[3.5] Implement inheritance, including abstract and sealed types as well as record classes. Override methods, including that of the Object class. Implement polymorphism and differentiate between object type and reference type. Perform reference type casting, identify object types using the instanceof operator, and pattern matching with the instanceof operator and the switch construct.	*Only pattern matching using the* `instanceof` *pattern match operator and the enhanced* `switch` *construct is covered in this chapter.*

Pattern matching allows testing of an object in order to extract data from the object if it matches a pattern. Although conditional extraction of data from an object has always been possible, using pattern matching results in code that is concise, safe, secure, and easier to read and maintain.

We take a deep dive into pattern matching for objects in Java 21 by exploring the following features of the language:

- Using *type patterns* with the `instanceof` *pattern match operator* (*finalized in Java 16)*
- Using *type patterns* with the *enhanced* `switch` *construct* (*finalized in Java 21*)
- Using *record patterns* with the `instanceof` *pattern match operator* (*finalized in Java 21*)
- Using *record patterns* with the *enhanced* `switch` *construct* (*finalized in Java 21*)

Along the way we use relevant code examples to illustrate pattern matching for objects and identify coding idioms that utilize patterns.

1.1 Type Pattern Matching with the `instanceof` Pattern Match Operator

The topic in this section may already be familiar, as it was made permanent in the language back in Java 16. However, it is included here in order to provide a comprehensive coverage of pattern matching for Java objects in one place.

The `instanceof` Type Comparison Operator

The binary `instanceof` operator can be used in two ways: as a *type comparison operator* and as a *pattern match operator*.

The `instanceof` *type comparison operator* has the following syntax for comparing types:

reference_expression `instanceof` *destination_type*

At runtime, the `instanceof` type comparison operator returns `true` if the left-hand operand (i.e., the reference value that results from the evaluation of *reference_expression*) can be a *subtype* of the right-hand operand (*destination_type*). It always returns `false` if the left-hand operand is `null`. If the `instanceof` operator returns `true`, the corresponding type cast expression `(` *destination_type* `)` when applied to the left-hand operand *reference_expression* is guaranteed to be safe:

`(` *destination_type* `)` *reference_expression*

The compiler only flags a compile-time error if it can conclusively determine that there can be no common subtype objects between the type of the *reference_expression* and the *destination_type*. A runtime check is necessary to determine whether the subtype relationship exists as mentioned earlier. If there is no such relationship between the types, then the cast `(` *destination_type* `)` *reference_expression* will be invalid, resulting in a `ClassCastException` if the cast is performed.

We assume the following type declarations in the discussion below.

Click here to view code image

```
interface IShape {}

sealed interface IPolygon extends IShape permits Triangle {}
final class Triangle implements IPolygon {}

class Circle implements IShape {}
```

Figure 1.1 shows the inheritance hierarchy for the types. The implementation of the types is kept simple for illustrative purposes.

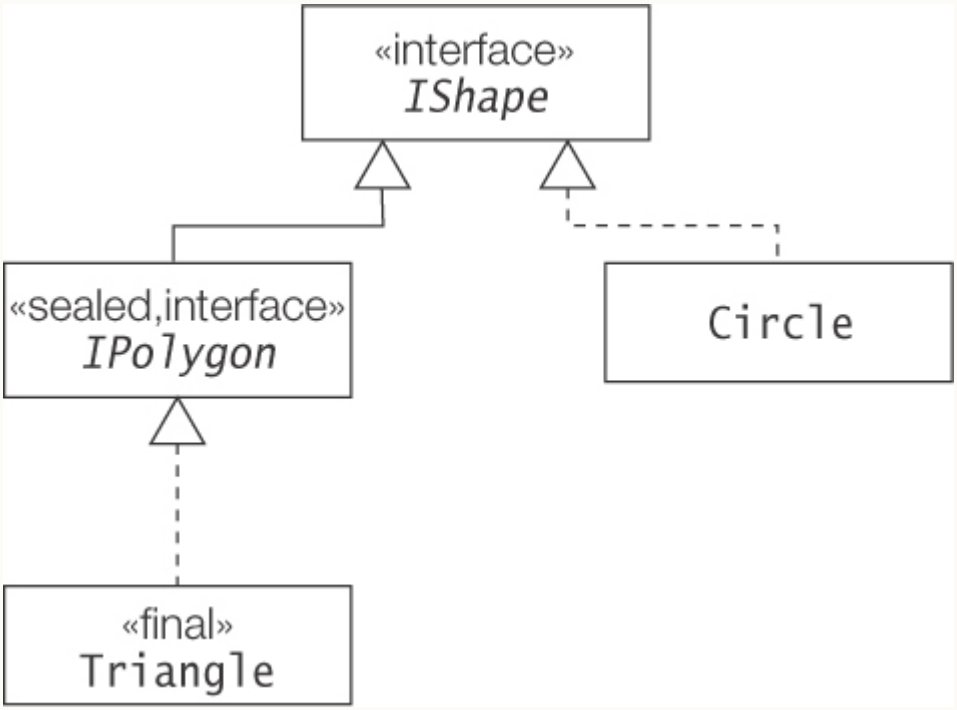


Figure 1.1 *Inheritance Hierarchy*

The *subtype-and-cast* idiom is commonly used to elicit subtype-specific behavior of an object referenced by a supertype reference. If the `instanceof` type comparison operator at (1a) returns `true`, the cast at (1b) can be safely applied and subtype-specific behavior can be elicited at (1c).

Click here to view code image

```
IShape ishape = new Triangle(); // Supertype reference denotes subtype object.

// Using the instanceof type comparison operator. (1)
if (ishape instanceof Triangle) {          // (1a) Correct subtype?
  Triangle triangle = (Triangle) ishape;   // (1b) Cast to subtype.
```

```
  System.out.println(triangle);          // (1c) Call subtype-specific method.
}
```

The `instanceof` Pattern Match Operator

Figure 1.2 shows the syntax of the `instanceof` operator when used as a *pattern match operator*: The `instanceof` pattern match operator augments the syntax of the `instanceof` type comparison operator with a *pattern variable*. The *type pattern* comprises a *local reference variable declaration* that is declared with a *destination type* and a single *pattern variable*. The `instanceof` pattern match operator essentially combines the `instanceof` type comparison operator with an implicit type cast and initialization of the *pattern variable*.



Figure 1.2 *The* `instanceof` *Pattern Match Operator*

At runtime, if the reference value of the object resulting from the evaluation of the *reference expression* can be *cast* to the *destination type* (i.e., the *dynamic type* of the resulting object is a *subtype* of the *destination type*), the object is said to *match* the *type pattern*. Only in this case does the `instanceof` pattern match operator return `true` and, as a side effect, *the reference value of the object is cast to the destination type and assigned to the pattern variable*. The *pattern variable* is never created or initialized if the `instanceof` pattern match operator returns `false`. It always returns `false` if the *reference expression* is `null`.

The *subtype-and-cast idiom* can be expressed more compactly with the `instanceof` pattern match operator as it type checks the object and casts its reference value that is assigned to the pattern variable, as illustrated below at (2).

Click here to view code image

```
IShape ishape = new Triangle(); // Supertype reference denotes subtype object.

// Using the instanceof pattern match operator.  (2)
if (ishape instanceof Triangle triangle) {      // Triangle subtype of IShape.
  System.out.println(triangle);
}
```

When the `instanceof` pattern match operator returns `true`, it is said to *introduce* a *pattern variable* into a well-defined scope, which in this case is the `if` block. The code above at (2) illustrates the case where the static type of the *reference expression* (`IShape`) is cast to a *narrower destination type* (`Triangle`). Generally, a runtime check is required in such cases to ensure that the *dynamic type* of the *reference expression* is assignment compatible to the type of the pattern variable.

The code below illustrates the case where the static type of the reference expression (`Triangle`) is implicitly cast to a *broader destination type* (`IShape`), where `IShape` is a supertype of `Triangle`. If the compiler can determine that a broader cast is required, then the runtime checks for assignment compatibility of the dynamic type of the reference expression to the type of the pattern variable is avoided, as broader reference casts are safe.

Click here to view code image

```
Triangle triangle = new Triangle();

if (triangle instanceof IShape ishape) {  // IShape is supertype of Triangle.
  System.out.println(ishape);
}
```

Valid Operand Types in the `instanceof` Pattern Match Operator

The types of the operands in the `instanceof` pattern match operator can be *unrelated*, unless the compiler determines that they are *inconvertible* (i.e., no object of one can be an object of the other type) or will result in an *unsafe cast* (i.e., compromise type safety).

We assume the following bare-bones type declarations in the discussion below:

Click here to view code image

```
interface Chart {}
class PieChart implements Chart {}            // Non-final class
final class Flowchart implements Chart {}      // Final class
```

Same Types as Operands

The operands of the `instanceof` pattern match operator can be of the same type, as shown at (1). This is equivalent to applying an *identity cast* to the object denoted by the

reference `c` and assigning it to the pattern variable `cc`.

Click here to view code image

```
Circle c = new Circle();

if (c instanceof Circle cc) {     // (1) OK.
  System.out.println(cc);
}
```

Unrelated Types as Operands

In the following method, although the `IShape` interface and the `PieChart` class are unrelated, they are allowed as operand types in the `instanceof` pattern match operator at (1).

Click here to view code image

```
static void unrelatedTypes(IShape s) {
  if (s instanceof PieChart p) {          // (1) Unrelated types allowed.
    System.out.println(p);
  } else {
    System.out.println("Not a PieChart");
  }
}
```

The method above will always print `Not a PieChart` when the parameter is a polygon referenced by `IShape`. Since the class `PieChart` is not `final`, a subclass can extend this class and also implement the `IShape` interface, as shown below. Such a subclass is both a `PieChart` and an `IShape`. At runtime, if the parameter referenced by `IShape` is an object of this subclass, the call to the method above would print the name of this class.

Click here to view code image

```
class ExplodedPieChart extends PieChart implements IShape {...}
```

Although the compiler will allow operands of unrelated types, it will flag an error if it can be determined that the types of the operands cannot have any values (except `null`) in common (i.e., are inconvertible). Such types are known as *disjoint types*.

Unrelated Classes as Operands

Classes that are unrelated by inheritance are disjoint classes. Since a class can only extend one superclass, such disjoint classes cannot be extended to have any common subclasses. In the code below, the classes `Circle` and `PieChart` are unrelated by inheritance and their references inconvertible, resulting in a compile-time error at (1).

Click here to view code image

```
Circle c = new Circle();

if (c instanceof PieChart pc) {     // (1) Compile-time error: Inconvertible types.
  System.out.println("true");
}
```

Peer Classes as Operands

Peer classes are also disjoint classes. They have a common supertype, but a reference of one peer type cannot be cast to the other. In the code below the peer classes `Triangle` and `Circle` have the interface `IShape` as their common supertype. A `Triangle` can never be a `Circle` and vice versa. Thus peer class references are inconvertible, and the code at (2) will not compile.

Click here to view code image

```
Triangle t = new Triangle();

if (t instanceof Circle c) {       // (2) Compile-time error.
  System.out.println("true");
}
```

Final Classes as Operands

A `final` class and a reference type that are unrelated by inheritance are also disjoint types. A `final` class cannot be extended, therefore no shared subtype can exist between the two types. In the code below `final` class `Flowchart` and the interface `IShape` are two such unrelated types, and the compiler will flag an error at (3).

Click here to view code image

```
Flowchart fc = new Flowchart();

if (fc instanceof IShape is) {     // (3) Compile-time error: Inconvertible types.
  System.out.println("true");
}
```

Sealed Types as Operands

The compiler also takes into consideration the sealed hierarchy to determine whether the operand types should be allowed. In particular, a reference of a sealed type whose permitted types are all declared `final` is not allowed as an operand type, if the other operand type is an unrelated class. In the code below, the sealed interface `IPolygon` only has the `final` class `Triangle` as its permitted class. A reference of any unrelated class, such as the class `PieChart`, and a reference of the sealed interface `IPolygon` are inconvertible types, resulting in a compile-time error at (4).

Click here to view code image

```
PieChart pc = new PieChart();

if (pc instanceof IPolygon p) {  // (4) Compile-time error: Inconvertible types.
  System.out.println("true");
}
```

Generic Types as Operands

The compiler can distinguish between the parameterized types `List<String>` and `List<Integer>`, but at runtime this is not possible as the erasure of `List<String>` and `List<Integer>` is just the raw type `List` with no information about the type arguments `String` and `Integer`. As discussed earlier, since the evaluation of the `instanceof` operator can involve a runtime check on the operand types, the compiler will not allow any operand types, including generic types, that are inconvertible or can result in an unsafe cast at runtime.

The discussion earlier of different kinds of operand types in the `instanceof` operator also applies to generic types. Parameterized generic types are allowed as oper- and types when a subtype relationship exists between them, as between `ArrayList<String>` and its supertype `List<String>` in (1) below. That is not the case at (2), since there is no such relationship between `ArrayList<String>` and `List<CharSequence>`, although `String` is a subtype of `CharSequence`. However, *unbounded generic types* (i.e., generic types param-

eterized with the *unbounded wild-card* (`?`)) or their raw types can be specified as the destination type in the `instanceof` operator, as in (3) and (4), respectively, since the implicit cast is to a broader destination type and therefore deemed safe.

Click here to view code image

```
ArrayList<String> list = new ArrayList<>();

if (list instanceof List<String> sl) {       // (1) Subtype relation. Safe cast.
  //...
}

if (list instanceof List<CharSequence> cl) {  // (2) Compile-time error!
  //...
}

if (list instanceof List<?> al) {            // (3) Unbounded wildcard allowed.
  //...
}

if (list instanceof List rl) {               // (4) Raw type allowed.
  //...
}
```

The above discussion also applies to type parameters as operand types in the `instanceof` operator, as shown at (1) in the generic method below.

Click here to view code image

```
static <T> void check(Object obj) {
  if (obj instanceof T t) {        // (1) Compile-time error! Unsafe cast.
    //...
  }
}
```

Technically, the `instanceof` operator only allows *reifiable types* as operands (i.e., a type whose type information is fully present at runtime: primitives, non-generic types, raw types, and generic types parameterized with only unbounded wildcards).

Properties of a Pattern Variable

A pattern variable *cannot* shadow another local variable by the same name—that is, it cannot be redeclared in a pattern if the local variable of the same name is still in scope.

Click here to view code image

```
// Local variables:
IShape ishape = new Circle();
Triangle triangle = new Triangle();     // (1)

if (ishape instanceof Triangle triangle) {// Error: triangle at (1) redeclared.
  System.out.println(triangle);
}
```

A pattern variable introduced by the `instanceof` pattern match operator can shadow a *field* of the same name in the class.

Click here to view code image

```
// Field declaration:
static IShape myShape;
...
// Local variable declaration:
IShape ishape = new Circle();
...
if (ishape instanceof Triangle myShape) {  // Local variable myShape introduced.
  System.out.println(myShape);             // Shadows field reference.
}
System.out.println(myShape);               // Uses field reference.
```

A pattern variable is not `final` in its scope unless it is declared `final`—in other words, the pattern variable can be declared with the modifier `final`, as shown at (1) below. Changing the value of a `final` pattern variable is flagged as a compile-time error, as shown at (2).

Click here to view code image

```
IShape ishape = new Circle();

if (ishape instanceof final Triangle triangle) {    // (1) final pattern variable.
  triangle = new Triangle();                        // (2) Compile-time error!
```

```
  System.out.println(triangle);
}
```

As we have seen, a pattern variable is analogous to a local variable, except that a local variable has block scope whereas a pattern variable has flow sensitive scope as explained next.

Flow Sensitive Scope of Pattern Variables

The scope of a pattern variable is the places in the code where the program control can reach only as a consequence of the `instanceof` pattern match operator returning `true`.

We first examine the scope of a pattern variable in an `if-else` statement. If the `instanceof` pattern match operator returns `true` in the conditional of an `if` statement, the pattern variable is introduced and its scope is the `if` block, as execution would continue in the `if` block. Not surprisingly, this is also the case for the `if-else` statement. In the code below, the pattern variable is *not* accessible in the `else` block when the `instanceof` pattern match operator returns `true`, since the `else` block would never be executed.

Click here to view code image

```
IShape ishape = new Triangle();

if (ishape instanceof Triangle triangle) {
  System.out.println(triangle);          // Pattern variable in scope.
} else {
  System.out.println(triangle);          // Compile-time error!
}
```

It is important to keep in mind that the `instanceof` pattern match operator introduces a pattern variable if and only if it returns `true`. In the `if-else` statement below, the `if` block is only executed if the conditional is `true`—that is, if the `instanceof` pattern match operator returns `false`. In that case, no pattern variable is introduced, and thus no pattern variable is ever accessible in the `if` block. However, if the conditional is `false`, the `instanceof` pattern match operator must be `true`, and thus introduces a pattern variable that is guaranteed to be accessible in the `else` block.

Click here to view code image

```
IShape ishape = new Triangle();

if (!(ishape instanceof Triangle triangle)) { // Logical complement operator (!)
  System.out.println(triangle);              // Compile-time error!
} else {
  System.out.println(triangle);              // Pattern variable in scope.
}
```

In fact, if the `if` block does *not complete normally* (e.g., by executing a `return`, a `break`, a `continue`, or a `throws` statement) as shown at (1) below, the pattern variable introduced is accessible in the `else` block and in code after the `if-else` statement.

Click here to view code image

```
IShape ishape = new Triangle();

if (!(ishape instanceof Triangle triangle)) {
  System.out.println("No triangle here");
  return;                                  // (1) Does not complete normally.
} else {
  System.out.println(triangle);            // Pattern variable in scope.
}
System.out.println(triangle);              // Pattern variable still in scope.
```

The `instanceof` pattern match operator can also introduce a pattern variable in certain *boolean expressions*. The conditional in the `if` statement below uses the conditional-AND operator (`&&`). The short-circuit evaluation of the `&&` operator ensures that the right-hand operand is only executed if the left-hand operand evaluates to `true`, thereby introducing the pattern variable that is then in scope in the right-hand operand of the conditional. The pattern variable will be in scope in the `if` block if the conditional evaluates to `true`—that is, both operands of the `&&` operator return `true`. In the code below, if the conditional is `true`, two pattern variables `triangle` and `poly` are introduced that are accessible in the `if` block. Applying the logical complement (`!`) operator to the conditional expression below works the same way as we have seen earlier with the `if-else` statement.

Click here to view code image

```
IShape ishape = new Triangle();

if (ishape instanceof Triangle triangle && triangle instanceof IPolygon poly) {
```

```
  System.out.println(triangle);
  System.out.println(poly);
}
```

The conditional-OR operator (`||`) does *not* introduce a pattern variable in the `if` block. Because of short-circuit evaluation of the `||` operator, the right-side operand of the conditional is only evaluated if the left-side operand is `false`, but then no pattern variable has been introduced by the `instanceof` pattern match operator in the left-side operand of the boolean expression.

Click here to view code image

```
IShape ishape = new Triangle();

if (ishape instanceof Triangle triangle ||
    triangle instanceof IPolygon poly) {    // Compile-time error.
  System.out.println(triangle);           // No guarantee. Compile-time error.
  System.out.println(poly);               // No guarantee. Compile-time error.
}
```

Using the same pattern variable (`triangle`) in the `instanceof` expressions below results in a compile-time error, as this is analogous to redeclaring a local variable.

Click here to view code image

```
IShape ishape = new Triangle();

if (ishape instanceof Triangle triangle &&  // Compile-time error!
    ishape instanceof IPolygon triangle) {  // Duplicate pattern variable triangle.
  System.out.println(triangle);
}
```

The `instanceof` pattern match operator can be used in the conditional (ternary) operator, and its semantics are analogous to those of the `if-else` statement:

Click here to view code image

```
IShape ishape = new Triangle();

String result = ishape instanceof Triangle triangle ?
```

```
    "Definitely a Triangle: " + triangle :
    "Certainly not a Triangle";
```

The `instanceof` pattern match operator can also introduce a pattern variable in loops—for example, when used in the conditional of `for(;;)` and `while` loops. This allows conditional processing of objects where loop termination is controlled by the `boolean` value of the `instanceof` pattern match operator and the pattern variable referring to the next object for processing.

Using the `instanceof` pattern match operator makes the code more concise and safer, as it combines three tasks into a single operation: subtype checking, reference casting, and assignment to a pattern variable. The class declaration below at (1) shows the idiom for implementing the `equals()` method for a class that is compact and safe, where corresponding fields of the objects must also be compared.

Click here to view code image

```
public class VersionNumber {
  private int release;
  private int revision;
  private int patch;
 // ...
  @Override
  public boolean equals(Object obj) {          // (1)
    return (this == obj)
        || (obj instanceof VersionNumber vno
           && this.patch  == vno.patch
           && this.revision == vno.revision
           && this.release  == vno.release);
  }
}
```

1.2 Type Pattern Matching with the Enhanced `switch` Construct

The `switch` construct implements a *multi-way branch* that allows program control to be transferred to a specific entry point in the code of the `switch` block based on a selector expression. Java has two variants of the `switch` construct: the `switch` *statement* and the `switch` *expression*. Each of them can be written in two different ways: Using the *colon* (`:`) *notation* and the *arrow* (`->`) *notation*.

Traditionally, only *constant expressions* were allowed as `case` constants in the `case` labels of a `switch` construct. The `case` constants could only be of the following types: Integral types (except `long`), their corresponding wrapper types, enum types, or `String`. The code to execute in the `switch` block is chosen based on an *equality test* on the value of the selector expression and the `case` constants in the order of their specification. We will use the term *traditional* `switch` *construct* to mean `switch` statements and `switch` expressions that have constant expressions in their `case` labels. Note that there are four ways of writing a traditional `switch` construct, as the `switch` statement and the `switch` expression can each be written using the colon notation and the arrow notation.

The `switch` construct has been enhanced to allow *patterns* in the `case` labels of a `switch` construct. The *enhanced* `switch` *construct* allows the value of the selector expression to be matched against patterns in the `case` labels, and the code associated with the `case` label whose pattern matches the selector expression is executed. Again, note that there are four ways of writing the enhanced `switch` construct.

A summary of the `switch` construct is shown in **Table 1.1**, that applies to both the traditional and the enhanced `switch` construct. **Table 1.2** summarizes how compile-time check for *exhaustiveness* (i.e., for all possible values of the selector expression, there must be a matching `case` label) and *fall-through* (i.e., whether execution can continue with the code associated with subsequent `case` labels) are handled in the traditional `switch` construct. We assume familiarity with the four variations of the traditional `switch` construct that allows constant expressions in `case` labels. Our focus here will be on the enhanced `switch` construct that allows patterns in `case` labels.

Table 1.1 *The* `switch` *Construct*

Notation	The `switch` Statement	The `switch` Expression
`case` *label* `:` *statements*	• Executes statements associated with the selected `case` label. • Only `break` and `return` statements allowed to control fall-through.	• Executes statements associated with the selected `case` label, but must have a `yield` statement to return a value. • No `break` or `return` statement allowed.

Notation	The `switch` Statement	The `switch` Expression
case *label* -> *action*	• Action associated with a switch rule can be one of the following: ○ *An expression statement* ○ *A block* ○ *A* throw *statement* • break and return statements allowed.	• Action associated with a switch rule can be one of the following: ○ *Any expression* ○ *A block* ○ *A* throw *statement* • No break or return statement allowed. • Must return a value that is either the value of a stand-alone *expression* or the value of the expression in a yield statement that can occur as the last statement in a *block*.

Table 1.2 *The Traditional* `switch` *Construct: Exhaustiveness and Fall-Through*

Notation	The Traditional `switch` Statement	The Traditional `switch` Expression
`case` *label* `:` *statements*	• No compile-time check for exhaustiveness. • Fall-through can occur.	• Compile-time check for exhaustiveness. • Fall-through can occur.
case *label* -> *action*	• No compile-time check for exhaustiveness. • *No* fall-through can occur.	• Compile-time check for exhaustiveness. • *No* fall-through can occur.

Using Type Patterns in `case` Labels

In an enhanced `switch` construct, the *selector expression* can be of *any reference type* and a `case` label can specify a *pattern*. Such a `case` label is known as a *pattern label* and the pattern specified in the `case` label is called a `case` *pattern*. The `case` pattern must be *compatible* with the selector expression.

If the pattern specified in the `case` label is a *type pattern*, it is called an *unguarded* `case` *pattern* or an *unconditional* `case` *pattern*, as opposed to a *guarded* `case` *pattern* or a *conditional* `case` *pattern* (p. 16). We first look at specifying unguarded type patterns in `case` labels.

The code below uses the `instanceof` pattern match operator as conditionals in a cascade `if-else` statement to compute the text representation of certain types of objects, where the reference `obj` of type `Object` denotes the object.

Click here to view code image

```
// Given the following record declaration:
record Point(double x, double y) {};

// Using type patterns in the instanceof pattern match operator:
Object obj = "Hello";
String result = null;
if (obj instanceof String str) {
  result = str.toUpperCase();
} else if (obj instanceof Point pnt) {
  result = pnt.toString();
} else if (obj instanceof int[] ia) {
  result = Arrays.toString(ia);
} else {
  result = "No match found.";
}
```

The code above is refactored to use the enhanced `switch` construct, as shown in Table 1.3. It shows the four variations of the enhanced `switch` construct. Apart from the `case` label with the `null` value and the `default` label, all other `case` labels specify type patterns.

Table 1.3 *The Enhanced* `switch` *Construct using Type Patterns*

The Enhanced `switch` Statement	The Enhanced `switch` Expression
Click here to view code image	Click here to view code image

// (1) Using colon notation in switch
// statement:
Object obj = "Hello";
String result = null;
switch (obj) {
  case null:
  result = "No object found."; break;
  case String str:
  result = str.toUpperCase(); break;
  case Point pnt:
  result = pnt.toString(); break;
  case int[] ia:
  result = Arrays.toString(ia); break;
  default:
  result = "No match found.";
}
	// (2) Using colon notation in switch
	// expression:
	Object obj = "Hello";
	String result = switch (obj) {
	  case null: yield "No object found.";
	  case String str: yield str.toUpperCase();
	  case Point pnt: yield pnt.toString();
	  case int[] ia: yield Arrays.toString(ia);
	  default: yield "No match found.";
	};

The Enhanced `switch` Statement	The Enhanced `switch` Expression
Click here to view code image	Click here to view code image

```
// (3) Using arrow notation in switch
//     statement:
Object obj = "Hello";
String result = null;
switch (obj) {
  case null ->
    result = "No object found.";
  case String str ->
    result = str.toUpperCase();
  case Point pnt  ->
    result = pnt.toString();
  case int[] ia  ->
    result = Arrays.toString(ia);
  default ->
    result = "No match found.";
}
```

```
// (4) Using arrow notation in switch
//     expression:
Object obj = "Hello";
String result = switch (obj) {
  case null       -> "No object found.";
  case String str -> str.toUpperCase();
  case Point pnt  -> pnt.toString();
  case int[] ia   -> Arrays.toString(ia);
  default         -> "No match found.";
};
```

The execution of the enhanced `switch` statement proceeds as follows:

- The selector expression is evaluated first, resulting in a reference to an object.
  If the selector expression evaluates to `null` and no `case` label with the `null` value is specified, a `NullPointerException` is thrown.
- The selector object is successively matched with the patterns in the `case` labels.
  As we would expect, control is transferred to the start of the code associated with the first `case` label whose pattern matches the selector object.
  The selector object trivially matches the `default` label if the `default` label is specified and no matches are found.
  An enhanced `switch` construct is always *exhaustive*—that is, the *type coverage* of all switch labels (i.e., either a `case` label or the `default` label) must include the type of the selector expression. The enhanced `switch` construct guarantees that at least one `case` label or the `default` label will match the selector object.

If the selector object matches the pattern in a `case` label, any pattern variables specified in the `case` pattern are introduced into this `case` label and its associated code. A type

pattern introduces a single pattern variable. Later we will see that a *record pattern* can introduce more than one pattern variable (p. 20).

Scope of the pattern variables introduced by a pattern is *flow sensitive* (p. 18).

The `null` Value as Case Constant in `switch` Construct

The `switch` construct allows the `null` value to be specified as a `case` constant so that appropriate action can be taken if the selector expression evaluates to `null`, thus avoiding the proverbial `NullPointerException` to be thrown implicitly.

Click here to view code image

```
Object obj = "Hello";
String result = switch (obj) {
  case null      -> "No object found";
  case String str -> str.toUpperCase();
  default        -> "No match found.";
};
```

If no `case` label with the `null` value is specified, it is implicitly implied—that is, a switch block without a `case` label with the `null` value is treated as if it has such a `case` label whose associated code throws a `NullPointerException`.

Click here to view code image

```
String newResult = switch (obj) {
...
  case null -> throw new NullPointerException();  // Implied if omitted.
...
};
```

A `case` label with the `null` value can occur in any order among the other switch labels. It can be combined with the `default` label, but not vice versa, as shown at (1) and (2). If such a combined label is specified, it must occur last, but then the `case` label with the `null` value or the `default` label cannot be used individually in the same `switch` construct, as shown at (3) and (4).

Click here to view code image

```
...
case null, default -> throw new NullPointerException(); // (1) OK.
case default, null -> throw new NullPointerException(); // (2) Compile-time error.
case null -> throw new NullPointerException(); // (3) Compile-time error due to (1)
default -> throw new NullPointerException();   // (4) Compile-time error due to (1)
...
```

Guarded Case Patterns

A *guarded case pattern* allows a pattern to be refined by a `boolean` or `Boolean` expression (called the *guard*) specified using an *optional* `when` *clause*. The code below determines whether an integer is an even or an odd number. A guarded case pattern is specified in (1) using a `when` clause whose operand is the `boolean` expression `ii % 2 == 0`.

Click here to view code image

```
Integer num = 1949;
String result1 = switch (num) {
  case Integer ii when ii % 2 == 0 -> "Even number: " + ii;  // (1)
  case Integer ii                  -> "Odd number: " + ii;   // (2)
};
```

A `case` label is selected for execution if the selector object matches the pattern in the `case` label *and* the associated `when` clause evaluates to `true`. In the code above, if the `case` label at (1) is not selected, the unguarded pattern at (2) will always be selected.

The scope of any pattern variable introduced by the pattern comprises the `when` clause and the code associated with the `case` label. It is a compile-time error if a `when` clause has a constant expression with the value `false`, as such a `case` label would never be selected.

Any variable that is used but not declared in a `when` clause must be either `final` or *effectively* `final`.

A guarded case pattern in the enhanced `switch` construct is equivalent to a conditional where the operands of the conditional-AND operator (`&&`) are a pattern in the `instanceof` pattern match operator and a `boolean` expression.

Click here to view code image

```
Object obj = 1949;
String result1 = null;
if (obj instanceof Integer ii && ii % 2 == 0) {
  result1 = "Even number: " + ii;
} else if (obj instanceof Integer ii) {
  result1 = "Odd number: " + ii;
}
System.out.println(result1);
```

Case Label Dominance

In a traditional `switch` statement at the most one `case` label can apply, since the selection of a switch label is based on the equality test. In an enhanced `switch` statement, several pattern labels can apply, as the value of the selector expression can be matched by several pattern labels.

Java uses the following heuristic to select a switch label: the first `case` label in the switch block that matches the selector expression is chosen. However, this requires *dominance analysis*, analogous to the `catch` blocks in a `try-catch-finally` construct. A pattern label results in a compile-time error if any previous pattern label or constant label *always* matches first—the former `case` label is said to *dominate* the latter.

In the code below, (2) will never be executed (i.e., it is *unreachable* code), as (1) will always be selected when the number is an `Integer`.

Click here to view code image

```
Integer num = 1949;
// Compile-time error: Wrong dominance order for case labels. (1) dominates (2).
String result2 = switch (num) {
  case Integer ii                -> "Odd number: " + ii;  // (1)
  case Integer ii when ii % 2 == 0 -> "Even number: " + ii; // (2)
};
```

The order of the `case` labels shown below will satisfy the dominance requirement in a `switch` block. In fact, the order in which any constant `case` label or guarded pattern `case` label is specified is irrelevant, as long as they are specified *before* any unguarded pattern `case` labels.

- Constant `case` labels (e.g., `case Integer.MAX_VALUE -> ...`)
- Guarded pattern `case` labels (e.g., `case Integer ii when ii % 2 == 0 -> ...`)

- Unguarded pattern `case` labels (e.g., `case Integer ii -> ..` .)

In the code below, the switch expression is exhaustive. The `case` labels cover all values of the selector expression. Note that a constant `case` label at (3) is specified after a guarded pattern `case` label at (2), but this is allowed as mentioned above. However, the two constant `case` labels and the guarded pattern `case` label are all specified before the unguarded pattern `case` label at (4). Note that the constant `case` label at (3) will never be executed.

Click here to view code image

```
Integer num = 10;
String result = switch (num) {
  case 0              -> "Number is 0";        // (1)
  case Integer ii
      when ii % 2 == 0 -> "Even number: " + ii;  // (2)
  case 10             -> "Number is 10";       // (3) Never executed.
  case Integer ii     -> "Odd number: " + ii;   // (4)
};
System.out.println(result);                    // (5) Even number: 10
```

The code above also illustrates that constant `case` labels and pattern `case` labels can be specified in the same `switch` construct.

Note the special case of a `when` clause whose constant expression evaluates to `true` . The `case` label at (5) will never be selected, as (4) will always be executed when the number is an `Integer` . However, changing the order of the `case` labels does not help, as (5) will dominate (4). Either (4) or (5) has to be omitted for the code to compile.

Click here to view code image

```
// Compile-time error: (4) dominates (5).
Integer num = 1949;
String result3 = switch (num) {
  case Integer ii when true -> "Integer value: " + ii; // (4)
  case Integer ii           -> "Number is " + ii;      // (5)
};
```

Flow Sensitive Scope of a Pattern Variable in `switch` Construct

Analogous to the flow sensitive scope of pattern variables introduced by an `instanceof` pattern match operator, the scope of a pattern variable introduced by a `case` label in an enhanced `switch` construct is the places in the code where the program control can reach only as a consequence of the `case` pattern matching the selector object. We summarize the scope of a pattern variable in a `case` label of the enhanced `switch` construct that can include the following code:

- The guard in the `when` clause of a guarded `case` label
- The action associated with a switch rule
- The statements associated with the `case` label of an enhanced `switch` statement

Exhaustiveness of Case Pattern Labels

Only the *traditional* `switch` *expression* with *constant expressions* in `case` labels requires that the `switch` block covers all possible values of the selector expression—that is, it must be *exhaustive*.

In contrast, the enhanced `switch` construct (both statement and expression) requires that the `switch` block is always exhaustive—that is, the *type coverage* of its switch labels is exhaustive. Type coverage is the union of types in the switch labels, and must include the type of the selector expression. In the code below, the union of types in the switch labels is `{Integer}` if no `default` label is specified, resulting in a compile-time error as the type of the selector expression `Number` is not covered by the union of the label types. Specifying a `default` label makes the `switch` block exhaustive, since the `default` label covers all types.

Click here to view code image

```
Number num = 1949;          // Superclass reference denotes a subtype object.
String result4 = switch (num) {
  case Integer ii when ii % 2 == 0 -> "Even number: " + ii; // (1)
  case Integer ii                  -> "Odd number: " + ii;  // (2)
  default          -> "Not an Integer"; // (3) Compile-time error if omitted.
};
```

It should be noted that, if a `default` label is specified, it can occur in *any order* among the switch labels specified in traditional `switch` construct (even first), but it is always specified *last* in enhanced `switch` construct.

A `switch` block *cannot* have more than one *match-all* switch label—that is, *either* the `default` label *or* an unguarded pattern `case` label that matches the selector expression. In the code below, if the `default` label at (2) is allowed, the `switch` statement will have a second match-all label in addition to the match-all `case` label at (1) that matches all strings. Either (1) or (2) is allowed, but not both.

Click here to view code image

```
static void translateResponse(String response) {
  switch (response.toUpperCase()) {
    case null       -> System.out.println("No response registered.");
    case "NEI"      -> System.out.println("Response is negative.");
    case "JA"       -> System.out.println("Response is affirmative.");
    case String s   -> System.out.println("Huh?");     // (1)
  // default        -> System.out.println("What?");    // (2) Compile-time error.
  }
}
```

Temptation to make the `switch` construct exhaustive by including a `default` label should be avoided. Best practice recommends that if possible, prefer an exhaustive `switch` construct that is without a match-all label.

Sealed Types and Exhaustiveness

Permitted types of a sealed type are taken into consideration to ensure exhaustiveness in an enhanced `switch` construct. The `switch` statement at (1) is exhaustive in the code below, as the `case` labels cover the inheritance hierarchy of the sealed type `BankCard` that is the type of the selector expression. The `default` label is allowed but redundant in this case. Omitting (2) or (3) will result in a compile-time error.

Click here to view code image

```
// A sealed type:
sealed interface BankCard permits DebitCard, CreditCard {}
final class DebitCard implements BankCard {}
final class CreditCard implements BankCard {}

static void whichCard(BankCard bc) {
  switch (bc) {                                  // (1)
    case null         -> System.out.println("No bank card");
    case DebitCard   dc -> System.out.println("It is " + dc);  // (2)
    case CreditCard  cc -> System.out.println("It is " + cc);  // (3)
```

```
//  default -> System.out.println("Huh?");        // (5) Redundant! Avoid.
 }
}
```

Illegal Fall-Through to a Pattern in `switch` Statement

No fall-through can occur in a `switch` construct that uses the arrow (`->`) notation, as the action associated with a switch rule terminates the execution of the `switch` construct. However, in an enhanced `switch` construct that uses the colon (`:`) notation, a fall-through to a `case` label that introduces pattern variables is flagged as a compile-time error.

Click here to view code image

```
Object obj = "Hello";
switch (obj) {
  case null     : System.out.println("No string found");  break;  // (1)
  case String str: System.out.println(str.toUpperCase());  break;  // (2)
  case Point  pnt: System.out.println(pnt.toString());     break;  // (3)
  default       : System.out.println("Unexpected value");        // (4)
}
```

Referring to line numbers in the code above:

1. If the `break` statement is omitted, execution will continue with the code associated with the `case` label in (2). This is *illegal* fall-through and is flagged as a compile-time error, as the `switch` body must not be executed if the selector expression is `null`.
2. If the `break` statement is omitted, execution will continue with the code associated with the `case` label in (3). This is *illegal* fall-through and is *not* flagged as a compile-time error, as the pattern variable `pnt` at (3) will not have been initialized.
3. If the `break` statement is omitted, execution will continue with the code associated with the `default` label in (4). This is *legal* fall-through and is flagged as a compile-time error, as the `default` label never introduces any pattern variables. One should be wary of such fall-through, as it may introduce logical errors.
   Even though the fall-through to the `default` label is allowed in this particular case, the pattern variable `pnt` cannot be accessed in the code associated with the `default` label—the scope of the pattern variable `pnt` excludes the `default` label.

The enhanced `switch` construct does not allow multiple patterns in a `case` label:

Click here to view code image

```
case Integer ii, String str -> ...          // Compile-time error!
```

Also, multiple `case` labels are not allowed in a switch label of the enhanced `switch` construct:

Click here to view code image

```
case Integer ii, case String str ->  ...      // Compile-time error!
```

However, a *list* of comma-separated *constant expressions* is allowed in a `case` label, regardless of which notation is used.

Table 1.4 summarizes the behavior of the four variations of the enhanced `switch` construct regarding exhaustiveness and fall-through. Compile-time checks ensure compliance regarding both exhaustiveness and fall-through.

Table 1.4 *The Enhanced* `switch` *Construct: Exhaustiveness and Fall-Through*

Notation	The Enhanced `switch` Statement	The Enhanced `switch` Expression
`case` *label* `:` *statements*	• Exhaustiveness mandatory. • No *illegal* fall-through allowed.	• Exhaustiveness mandatory. • No *illegal* fall-through allowed.
`case` *label* `->` *action*	• Exhaustiveness mandatory. • No fall-through allowed.	• Exhaustiveness mandatory. • No fall-through allowed.

1.3 Record Patterns

In addition to using type patterns for pattern matching of objects, we can also use *record patterns* that facilitate conditional extraction of component values from records—that is,

use record patterns for *record deconstruction*.

A record pattern specifies *the record type* and *an optional list of patterns* in which each pattern can be *a type pattern* or *a record pattern*. In other words, record patterns can be nested to match the component hierarchy of nested records. A record pattern allows a record to be matched against a record type and binds pattern variables in its pattern list to the components of the record, if a match is found. The list of patterns must match with the list of components in the record type.

In the code below, a record class named `PhoneNumber` at (1) is declared with three components. The `instanceof` pattern match operator at (2) uses the record pattern `PhoneNumber(int cc, int ac, int ln)` to match a `PhoneNumber`. At runtime, since the record referenced by `obj` is a `PhoneNumber`, the record pattern matches the record. The pattern variables `cc`, `ac`, and `ln` are assigned the values of the country code, the area code, and the local number components by calling the corresponding accessor methods in the record, respectively. The pattern variables introduced are then accessible in the `if` block.

Click here to view code image

```
record PhoneNumber(int countryCode, int areaCode, int localNumber) { }    // (1)
...
Object obj = new PhoneNumber(1, 800, 1234567);

if (obj instanceof PhoneNumber(int cc, int ac, int ln)) {              // (2)
  System.out.printf("%d-%d-%d%n", cc, ac, ln);
}
```

The pattern list in the record pattern `PhoneNumber(int cc, int ac, int ln)` at (2) consists of three *nested type patterns:* `int cc`, `int ac`, and `int ln`. The names of the pattern variables need not be the same as the component names in the record declaration. Note also that the destination type of these nested type patterns is the primitive type `int`—in contrast to the destination type of a *top-level type pattern*, that must always be a *reference type*.

A record pattern can be used in contexts where a top-level type pattern can be used, for example, as an operand of the `instanceof` pattern match operator (p. 22) and in the `case` labels of an enhanced `switch` construct (p. 26).

An enhanced `for(:)` loop requires a *local variable declaration* of a loop variable in its header to successively denote the elements to iterate over. A record pattern is *not* a local variable declaration, and therefore cannot be specified in the header of an enhanced `for(:)` loop, as shown at (1). However, the local variable declaration in the header of the enhanced `for(:)` loop at (2) may be considered a top-level *type pattern*, as a type pattern is syntactically equivalent to a local variable declaration.

Click here to view code image

```
record Item(int value) {}
Item[] items = {new Item(10), new Item(20), new Item(30)};

for (Item(int v) : items) {              // (1) Record pattern not allowed!
  System.out.println(v);
}

Integer[] intArray = {10, 20, 30, 40};
for (Integer n : intArray) {             // (2) Local variable declaration.
  System.out.println(n);
}
```

As expected, the `null` value does not match any record pattern. If pattern matching completes abruptly—for example, when an assessor method throws an exception—a `MatchException` is thrown to indicate an unexpected failure (p. 35).

The syntax of a record pattern is shown in Figure 1.3.

record_pattern: record_type ([list_of_patterns])

list_of_patterns: pattern { , pattern }

pattern: type_pattern | record_pattern

type_pattern: local_variable_type pattern_variable

Figure 1.3 *Record Patterns*

1.4 Record Pattern Matching with the `instanceof` Pattern Match Operator

Example 1.1 illustrates using record patterns with the `instanceof` pattern match operator. Using a type pattern to deconstruct a record requires calling assessor methods to extract the component values, as illustrated by the code at (1). Using a record pattern to

deconstruct a record makes the code more concise and safer, as the pattern variables in the record pattern refer to the component values, as illustrated by the code at (2).

Example 1.1 *Using Record Patterns in the* `instanceof` *Pattern Match Operator*

Click here to view code image

```
package record_patterns;

public class PhoneUser {

  static void printPhoneNumber(Object obj) {

    // Type pattern:                                    (1)
    if (obj instanceof PhoneNumber p) {
      int cc = p.countryCode();
      int ac = p.areaCode();
      int ln = p.localNumber();
      System.out.printf("%d-%d-%d%n", cc, ac, ln);

    }

    // Record pattern:                                  (2)
    if (obj instanceof PhoneNumber(int cc, int ac, int ln)) {
      System.out.printf("%d-%d-%d%n", cc, ac, ln);
    }

    // Record pattern using var:                        (3)
    if (obj instanceof PhoneNumber(var cc, var ac, var ln)) {
      System.out.printf("%d-%d-%d%n", cc, ac, ln);
      }

      // Refined record pattern:                        (4)
      if (obj instanceof PhoneNumber(var cc, var ac, var ln)
          && !String.valueOf(ac).startsWith("900")) {
        System.out.printf("%d-%d-%d%n", cc, ac, ln);
      } else {
        System.out.println("900-number cannot be revealed.");
      }
    }

    public static void main(String[] args) {
      PhoneNumber pn5 = new PhoneNumber(1, 900, 2571038);
      printPhoneNumber(pn5);
```

```
    }
  }
```

Output from the program:

Click here to view code image

```
1-900-2571038
1-900-2571038
1-900-2571038
900-number cannot be revealed.
```

Type Inference of Pattern Variables using `var`

Type inference of pattern variables in record patterns can be performed using `var`—that is, the type of the corresponding component in the record need not be specified for a type pattern in a record pattern. The code at (3) in **Example 1.1** illustrates the use of `var` for type inference of pattern variables, where the type of the pattern variables is inferred to be `int`.

Flow Sensitive Scope of Pattern Variables in Record Patterns

The flow sensitive scope of the pattern variables in a record pattern is analogous to that of the pattern variable in a type pattern.

An example of such scope is shown at (4) in **Example 1.1**, where the pattern variables are only accessible in the second operand of the conditional-AND (&&) operator in the `if` conditional and the `if` block, if the `instanceof` pattern match operator returns `true`.

Nested Record Patterns

As record patterns can be nested, they facilitate data navigation of the component hierarchy of nested records, which are matched recursively. The nested record class `Contract` at (1) has a record (`PhoneNumber`) as a component. The nested record pattern at (2) allows the component hierarchy of a `Contact` record to be deconstructed without resorting to assessor methods.

Click here to view code image

```
record Contact(String name, PhoneNumber phoneNumber) { }              // (1)
...
Object obj = new Contact("Java Guru", new PhoneNumber(1, 800, 1234567));

if (obj instanceof
      Contact(String name, PhoneNumber(int cc, int ac, int ln))) {      // (2)
  System.out.printf("%s: %d-%d-%d%n", name, cc, ac, ln);
}
```

We can of course make use of type inference of pattern variables using `var` in nested record patterns, as in (2a), instead of explicitly specifying the destination type of each type pattern in the pattern list:

Click here to view code image

```
if (obj instanceof
      Contact(var name, PhoneNumber(var cc, var ac, var ln))) {     // (2a)
  System.out.printf("%s: %d-%d-%d%n", name, cc, ac, ln);
}
```

Using Record Patterns with Generic Types

We can specify record patterns for generic record types. Below at (1) is a declaration of a generic record type, `Pair<T>`, whose two components are of the same type parameter `T`. A record of this type, parameterized with the type argument `String`, is constructed at (2).

In a record pattern, the generic record type can be specified with or without the type arguments. In the latter case, the record type is inferred. The generic record pattern in (3) is explicitly parameterized with the type argument `String`, whereas in (4) and (5) the record type is the unbounded record type `Pair<?>` and the raw record type `Pair`, respectively. The pattern variable types in (4) and (5) are inferred to be of type `String` from the record reference `strPair` in the `instanceof` pattern match operator. In fact, any record of type `Pair<T>` will match the record patterns in (4) and (5), where the types of the pattern variables is inferred from the record resulting from the evaluation of the reference expression.

Click here to view code image

```
record Pair<T>(T item1, T item2) {}                                  // (1)
...
Pair<String> strPair = new Pair<>("Milk", "Honey");                  // (2)

// With parameterization.
if (strPair instanceof Pair<String>(String a1, String a2)) {         // (3)
  System.out.println("Pair1: " + a1 + ", " + a2);
}

// With unbounded wildcard and using var.
if (strPair instanceof Pair<?>(var a1, var a2)) {                    // (4)
  System.out.println("Pair2: " + a1 + ", " + a2);
}

// Without parameterization and using var.
if (strPair instanceof Pair(var a1, var a2)) {                       // (5)
  System.out.println("Pair3: " + a1 + ", " + a2);
}

// Diamond operator:
if (strPair instanceof Pair<>(var a1, var a2)) {                     // (6) Compile-time error!
  System.out.println("Pair4: " + a1 + ", " + a2);
}
```

Parameterizing the record type with the diamond (`<>`) operator results in a compile-time error, as shown above at (6). This operator is used in the context of calling a constructor and not in record patterns.

Whereas a generic record type can either be explicitly specified with type arguments or implicitly inferred when omitted in a record pattern, this is not the case for a generic type pattern. In a generic type pattern, the explicit type arguments are allowed, and if omitted, are not inferred but treated as raw types (p. 7).

An object is compatible with a record pattern if it can be cast to the record type of the pattern without requiring an unchecked conversion—that is, making sure that the type safely of the cast is not compromised. Given the record `intPair` at (1), it is used as the reference expression in the `instanceof` pattern match operator at (2) and (3). In (2) we see that the type of the reference expression `intPair` is `Pair<Integer>` that cannot be safely cast to the type `Pair<String>` of the record pattern—the code will not compile. In (3) we see that the type `Pair<Integer>` of the reference expression `intPair` is a subtype of the type `Pair<?>` of the record pattern—the code will compile, but the `instanceof` pattern match operator will return `false` at runtime, as the two `Integer` components of

the record `intPair` are not compatible with the pattern list `(String a1, String a2)` of the record pattern.

Click here to view code image

```
Pair<Integer> intPair = new Pair<>(100, 100);              // (1)

if (intPair instanceof Pair<String>(String a1, String a2)) {// (2) Compile-time
                                                           //  error!
  System.out.println("Pair5: " + a1 + ", " + a2);
}

if (intPair instanceof Pair<?>(String a1, String a2)) {     // (3) false
  System.out.println("Pair6: " + a1 + ", " + a2);
}
```

The code below for matching the raw record at (4) to a parameterized record pattern at (5) will not compile, as the raw record type `Pair` cannot be safely cast to the parameterized record type `Pair<String>` of the record pattern. However, the raw record type `Pair` is a subtype of the unbounded record pattern `Pair<?>` at (6), so code will compile, but at runtime, the `instanceof` pattern match operator will return `false`, as the two `Integer` components of the record `intPair` are not compatible with the pattern list `(String a1, String a2)` of the unbounded record pattern.

Click here to view code image

```
Pair rawPair = new Pair(10, 20);                           // (4)

if (rawPair instanceof Pair<String>(String a1, String a2)) {       // (5) Compile-time
                                                           //     error!
  System.out.println("Pair7: " + a1 + ", " + a2);
}

if (rawPair instanceof Pair<?>(String a1, String a2)) {           // (6) false
  System.out.println("Pair8: " + a1 + ", " + a2);
}
```

1.5 Record Pattern Matching with the Enhanced `switch` Construct

Using record patterns in `case` labels is analogous to using type patterns in `case` labels of an enhanced `switch` construct (p. 11). As in the case of type patterns, the enhanced

`switch` construct must be exhaustive when record patterns are specified in `case` labels —that is, the switch body must provide switch labels to cover all possible values of the selector object. The type of the selector expression in the `switch` statement below at (1) is the record type `PhoneNumber`. The record pattern in the `case` label at (2) will match any record whose type is `PhoneNumber`. The `switch` statement at (1) isthus exhaustive.

Click here to view code image

```
// Record declaration:
record PhoneNumber(int countryCode, int areaCode, int localNumber) { }
...
PhoneNumber pn1 = new PhoneNumber(1, 607, 2571038);

switch (pn1) {                                    // (1) Exhaustive.
  case null -> System.out.printf("No phone number found.");
  case PhoneNumber(int cc, int ac, int ln) -> {              // (2)
    String pnToString = String.format("%d-%d-%d", cc, ac, ln);
    System.out.printf("Phone number: %s%n", pnToString);
  }
}
```

However, the type of the selector expression in the `switch` statement below at (3) is `Object`. The switch labels must cover all objects that an `Object` reference can denote, not just `PhoneNumber` records. The `default` label at (6) is necessary to make the `switch` statement exhaustive.

Click here to view code image

```
Object obj = pn1;               // (3) Supertype reference to subtype object.

switch (obj) {                  // (4) Exhaustive if default label specified.
  case null -> System.out.printf("No object found.");
  case PhoneNumber(int cc, int ac, int ln) -> {         // (5)
    String pnToString = String.format("%d-%d-%d", cc, ac, ln);
    System.out.printf("Phone number: %s%n", pnToString);
  }
  default ->  System.out.println("Not a phone number."); // (6) Necessary.
}
```

Nested Record Patterns in `case` Labels

Analogous to nested record patterns in the `instanceof` pattern match operator (p. 23), such patterns can also be specified in the `case` labels of an enhanced `switch` construct. The nested record class `Contract` at (1) has a record (`PhoneNumber`) as a component. The nested record pattern at (2) deconstructs the component hierarchy of a `Contact` record safely and concisely.

Click here to view code image

```
record Contact(String name, PhoneNumber phoneNumber) { }              // (1)
...
Object obj = new Contact("Java Guru", new PhoneNumber(1, 800, 1234567));

switch (obj) {
  case Contact(String name, PhoneNumber(int cc, int ac, int ln)) -> {     // (2)
    System.out.printf("%s: %d-%d-%d%n", name, cc, ac, ln);
  }
  default -> System.out.println("No match found.");
}
```

Guarded Record Patterns in `case` Labels

Analogous to guarded type patterns in `case` labels (p. 16), record patterns in an enhanced `switch` construct can be refined using the optional `when` clause. A `case` label is selected for execution if the selector object matches the record pattern in the `case` label *and* the associated `when` clause evaluates to `true`. Such a guarded case record pattern is used at (1) below to select any 800 phone numbers. If the `case` label at (1) is not selected, the unguarded pattern at (2) will always be selected.

Click here to view code image

```
PhoneNumber pn = new PhoneNumber(1, 800, 5551038);

switch (pn) {
  case null -> System.out.printf("No phone number found.");
  case PhoneNumber(var cc, var ac, var ln) when ac == 800 -> {  // (1)
    System.out.println("800 phone numbers cannot be listed.");
  }
  case PhoneNumber(var cc, var ac, var ln) -> {               // (2)
    String pnToString = String.format("%d-%d-%d", cc, ac, ln);
    System.out.printf("Phone number: %s%n", pnToString);
```

```
  }
}
```

Flow Sensitive Scope of Pattern Variables in the `switch` Construct

Analogous to the flow sensitive scope of type patterns in `case` labels (p. 18), the scope of any pattern variable introduced by a record pattern comprises the `when` clause and the code associated with the `case` label.

Type Inference of Record Pattern Variables in `case` Labels

Analogous to the type inference of pattern variables with `var` in a record pattern used in the `instanceof` pattern match operator (p. 23), such type inference is also performed for pattern variables declared with `var` in record patterns used in `case` labels. The pattern variables in record patterns at (1) and (2) above are declared with `var` for type inference of the pattern variables, where the type of the pattern variables is inferred to be `int`.

Case Label Dominance in Record Patterns

The discussion on case label dominance for type patterns in `case` labels also applies to record patterns in `case` labels (p. 16). Reversing the order of the `case` labels (1) and (2) above will result in the unguarded record pattern to dominate the guarded record pattern, resulting in a compile-time error.

Illegal Fall-Through in `case` Labels with Record Patterns

The compiler alerts to any illegal fall-through in an enhanced `switch` construct with record patterns in `case` labels (p. 19). Fall-through to the `default` label is allowed in an enhanced `switch` statement that uses the colon (`:`) notation.

Record patterns facilitate marshalling of component values of records as illustrated in Example 1.2. The example shows how record patterns can be used to process records in a stream.

Example 1.2 *Using Record Patterns in Streams*

Click here to view code image

```
package record_patterns;

record PhoneNumber(int countryCode, int areaCode, int localNumber) { }
```

```
record Contact(String name, PhoneNumber phoneNumber) { }
```

```
package record_patterns;
import java.util.Arrays;

public class StreamPhoneUser {
  public static void main(String[] args) {

    // Some ready-made contacts:
    Contact cn0 = new Contact("Tom",     new PhoneNumber( 1,  607, 1234567));
    Contact cn1 = new Contact("Dick",    new PhoneNumber( 1,  800, 4444444));
    Contact cn2 = new Contact("Harriet", new PhoneNumber(49,   30, 987654321));

    // An array of contacts.
    Contact[] cnArray = {cn0, cn1, cn2};

    System.out.println("Name  CC  AC  LPN");
    // Process the contacts:
    Arrays.stream(cnArray)                                  // (1)
        .map(cn ->                                          // (2)
            switch (cn) {                                   // (3)
              case Contact(var name, PhoneNumber(var cc, var ac, var ln))
                -> String.format("%-10s%-4d%-4d%-8d", name, cc, ac, ln);
            }
          )
          .forEach(str -> System.out.printf("%s%n", str));          // (4)
  }
}
```

Output from the program:

Click here to view code image

```
Name     CC  AC  LPN
Tom      1   607 1234567
Dick     1   800 4444444
Harriet  49  30  987654321
```

Using Sealed Types with Record Patterns in `case` Labels

Permitted types of a sealed type are taken into consideration to ensure exhaustiveness in an enhanced `switch` construct. Analogous to using sealed types with type patterns in the enhanced `switch` construct (p. 19), using sealed types with record patterns in the enhanced `switch` construct only requires `case` labels with record patterns of the permitted subtypes in order to be exhaustive.

Permitted types of the sealed type `TheatreTicket` below are record types. The `switch` statement at (1) is exhaustive in the code below, as the `case` labels cover the inheritance hierarchy of the sealed type `TheatreTicket` that is the type of the selector expression. Omitting (2), (3) or (4) will result in a compile-time error.

Click here to view code image

```
sealed interface TheatreTicket permits Balcony, MiddleSection, LowerSection {}
record Balcony(double price, int row, int num) implements TheatreTicket {}
record MiddleSection(double price, int row, int num) implements TheatreTicket {}
record LowerSection(double price, int row, int num) implements TheatreTicket {}
...
static void whichSeat(TheatreTicket tt) {
  switch (tt) {                                          // (1)
    case null -> System.out.println("No theatre ticket.");
    case Balcony(var price, var row, var num) ->                    // (2)
      System.out.printf("Balcony: $%.2f, [%d, %d]%n", price, row, num);
    case MiddleSection(var price, var row, var num) ->              // (3)
      System.out.printf("Middle section: $%.2f, [%d, %d]%n", price, row, num);
    case LowerSection(var price, var row, var num) ->               // (4)
      System.out.printf("Lower section: $%.2f, [%d, %d]%n", price, row, num);
  }
}
```

The method call `whichSeat(ticket)`, where ticket is declared as follows:

Click here to view code image

```
TheatreTicket ticket = new Balcony(20.99, 4, 10);
```

will print:

```
Balcony: $20.99, [4, 10]
```

Using Generic Record Patterns in `case` Labels

We can specify record patterns for generic record types in the `case` labels of an enhanced `switch` construct—analogous to record patterns for generic record types in the `instanceof` pattern match operator (p. 24).

Parameterized record type as reference type of the selector expression requires `case` labels that make the `switch` construct exhaustive. Below at (1) is a declaration of a generic record type, `Pair<T>`, whose two components are of the same type parameter `T`. A record of this type, parameterized with the type argument `String`, is constructed at (2). The generic record pattern in (3) is explicitly parameterized with the type argument `String`, whereas in (4) and (5) the record type is the unbounded record type `Pair<?>` and the raw record type `Pair`, respectively. The pattern variable types in (4) and (5) are inferred to be of type `String` from the record reference `strPair` that is the selector expression. In fact, any record of type `Pair<T>` will match the record patterns in (4) and (5), where the types of the pattern variables are inferred from the record type of the selector expression.

Click here to view code image

```
record Pair<T>(T item1, T item2) {}                    // (1)
...
Pair<String> strPair = new Pair<>("Milk", "Honey");       // (2)

// With parameterization.
switch(strPair) {                                      // (3)
  case Pair<String>(String a1, String a2) ->
    System.out.println("Pair1: " + a1 + ", " + a2);
}

// With unbounded wildcard and using var.
switch(strPair) {
  case Pair<?>(var a1, var a2) ->                      // (4)
    System.out.println("Pair2: " + a1 + ", " + a2);
}

// Without parameterization and using var.
switch(strPair) {
  case  Pair(var a1, var a2) ->                        // (5)
    System.out.println("Pair3: " + a1 + ", " + a2);
}

// Diamond operator:
```

```
switch(strPair) {
  case  Pair<>(var a1, var a2) ->                  // (6) Compile-time error!
    System.out.println("Pair4: " + a1 + ", " + a2);
}
```

Parameterizing the record type with the diamond (`<>`) operator results in a compile-time error, as shown above at (6), as this operator is not allowed in this context.

The selector object is compatible with a record pattern if it can be cast to the record type of the pattern without requiring an unchecked conversion—meaning that the type safety of the cast is not compromised. Given the record `intPair` at (1), it is used as the selector object in the enhanced `switch` construct at (2) and (3). In (2) we see that the type of the selector expression `intPair` is `Pair<Integer>` that cannot be safely cast to the type `Pair<String>` of the record pattern—the code will not compile. In (3) we see that the type `Pair<Integer>` of the selector expression `intPair` is a subtype of the type `Pair<?>` of the record pattern, but the record pattern at (3) only covers pairs with the pattern list `(String a1, String a2)` . The `switch` statement is therefore not exhaustive. One way to make it exhaustive is to use the record pattern in (3a) that will be matched for all pairs that do not have strings.

Click here to view code image

```
Pair<Integer> intPair = new Pair<>(100, 100);         // (1)

switch(intPair) {
  case Pair<String>(String a1, String a2) ->          // (2) Compile-time error!
  System.out.println("Pair5: " + a1 + ", " + a2);
}

switch(intPair) {
  case Pair<?>(String a1, String a2) ->               // (3)
    System.out.println("Pair6: " + a1 + ", " + a2);
  case Pair<?>(var a1, var a2) ->                     // (3a)
    System.out.println("Not a Pair of strings.");
}
```

The code below for matching the raw record at (4) to a parameterized record pattern at (5) will not compile, as the raw record type `Pair` cannot be safely cast to the parameterized record type `Pair<String>` of the record pattern. However, the raw record type `Pair` is a subtype of the unbounded record type `Pair<?>` , and the pattern list is allowed, but the `switch` statement is not exhaustive. The record pattern at (7) matches all pairs

whose components are of type `Integer`, as the component types of the selector object are now compatible with the pattern variables at (7). However, it is the record pattern at (8) that makes the `switch` statement exhaustive, as the component types of *any* selector pair are now compatible with the pattern variables of type `Object` at (8).

Click here to view code image

```
Pair rawPair = new Pair(10, 20);                          // (4)

switch(rawPair) {
  case Pair<String>(String a1, String a2) ->              // (5) Compile-time error!
    System.out.println("Pair7: " + a1 + ", " + a2);
}

switch(rawPair) {
  case Pair<?>(String a1, String a2) ->                   // (6)
    System.out.println("Pair8: " + a1 + ", " + a2);
  case Pair<?>(Integer a1, Integer a2) ->                 // (7)
    System.out.println("Pair9: " + a1 + ", " + a2);
  case Pair<?>(Object a1, Object a2) ->                   // (8) Exhaustive.
    System.out.println("Pair10: " + a1 + ", " + a2);
}
```

Using Generic Record Patterns parameterized with Sealed Types

Finally we present an example to illustrate how an enhanced `switch` construct can be made exhaustive when the selector object is of a generic record type that is parameterized with a sealed type. The generic record type `Pair<T>` is parameterized with the sealed type `BankCard` in **Example 1.3** to illustrate different scenarios that make the enhanced `switch` construct exhaustive.

Following comments refer to the numbered code lines in **Example 1.3**:

1. Declaration of the generic record type `Pair<T>` whose elements have the same type.
2. Declarations of the sealed interface `BankCard` and its `final` permitted subclasses `DebitCard` and `CreditCard`. Record patterns are created with the generic record type `Pair<T>` that are parameterized with the permitted record types of the sealed interface `BankCard`.
3. The `switch` expression specifies four `case` labels where each record pattern represents one of the four permutations for creating pairs with two permitted subclasses `DebitCard` and `CreditCard` of the sealed type `BankCard`. These four `case` labels together make the `switch` expression exhaustive.

4. The record pattern `Pair<BankCard>(BankCard b1, BankCard b2)` at (4a) in the `case` label of the enhanced `switch` expression is sufficient to cover any pair created with the two permitted subclasses `DebitCard` and `CreditCard`. This `case` label alone makes the `switch` expression exhaustive. Alternatively, (4b) can be used instead of (4a). The record pattern `Pair<?>(var b1, var b2)` in (4b) is more general, where any pair is a subtype of `Pair<?>` and the types of the pattern variables are inferred by use of `var`. Thus, any pair parametrized by any type will match this record pattern.
5. The `switch` expression specifies three `case` labels. The `case` labels at (5a) and (5b) use guarded record patterns to match pairs whose components have the same permitted subtype, either `DebitCard` or `CreditCard`. Pairs whose components have different permitted subtypes are matched by the `case` label at (5c). These three `case` labels together make the `switch` expression exhaustive.

Example 1.3 *Using Generic Record Patterns Parameterized with Sealed Types*

Click here to view code image

```
package record_patterns;
// A generic pair.
record Pair<T>(T item1, T item2) {}                    // (1)
```

```
package record_patterns;
// Sealed type BankCard hierarchy
sealed interface BankCard permits DebitCard, CreditCard {}    // (2)
final class DebitCard  implements BankCard {}
final class CreditCard implements BankCard {}
```

```
package record_patterns;

public class GenericRecordPatternsParametrizedWithSealedTypeInSwitch {

  static void identifyCards(Pair<BankCard> pair) {
    System.out.println("-".repeat(75));

    String result1 = switch (pair) {                          // (3)
      case Pair<BankCard>(CreditCard c1, CreditCard c2) -> c1 + ", " + c2;
      case Pair<BankCard>(DebitCard  d1, DebitCard  d2) -> d1 + ", " + d2;
      case Pair<BankCard>(CreditCard c,  DebitCard  d)  -> c  + ", " + d;
      case Pair<BankCard>(DebitCard  d,  CreditCard c)  -> d  + ", " + c;
    };
    System.out.println("1: " + result1);
```

```
    String result2 = switch (pair) {                          // (4)
      case Pair<BankCard>(BankCard b1, BankCard b2) -> b1 + ", " + b2;   // (4a)
//    case Pair<?>(var b1, var b2) -> b1 + ", " + b2;                 // (4b)
    };
    System.out.println("2: " + result2);

    String result3 = switch (pair) {                          // (5)
      case Pair<BankCard>(CreditCard cc, BankCard bc)                 // (5a)
           when (bc instanceof CreditCard cc2) -> "Two credit cards";
      case Pair<BankCard>(DebitCard dc, BankCard bc)                  // (5b)
           when (bc instanceof DebitCard dc2)  -> "Two debit cards";
      case Pair<BankCard>(BankCard b1, BankCard b2)  -> b1 + ", " + b2;  // (5c)
    };
    System.out.println("3: " + result3);
  }

  public static void main(String[] args) {
    Pair<BankCard> pair1 = new Pair<>(new DebitCard(),  new CreditCard());
    Pair<BankCard> pair2 = new Pair<>(new CreditCard(), new DebitCard());
    Pair<BankCard> pair3 = new Pair<>(new CreditCard(), new CreditCard());
    Pair<BankCard> pair4 = new Pair<>(new DebitCard(),  new DebitCard());

    identifyCards(pair1);
    identifyCards(pair2);
    identifyCards(pair3);
    identifyCards(pair4);
  }
}
```

Output from the program:

Click here to view code image

```
--------------------------------------------------------------------------
1: record_patterns.DebitCard@27c170f0, record_patterns.CreditCard@5451c3a8
2: record_patterns.DebitCard@27c170f0, record_patterns.CreditCard@5451c3a8
3: record_patterns.DebitCard@27c170f0, record_patterns.CreditCard@5451c3a8
--------------------------------------------------------------------------
1: record_patterns.CreditCard@5305068a, record_patterns.DebitCard@1f32e575
2: record_patterns.CreditCard@5305068a, record_patterns.DebitCard@1f32e575
3: record_patterns.CreditCard@5305068a, record_patterns.DebitCard@1f32e575
--------------------------------------------------------------------------
1: record_patterns.CreditCard@5451c3a8, record_patterns.CreditCard@5305068a
```

```
2: record_patterns.CreditCard@5451c3a8, record_patterns.CreditCard@5305068a
3: Two credit cards
--------------------------------------------------------------------------------
1: record_patterns.DebitCard@27c170f0, record_patterns.DebitCard@1f32e575
2: record_patterns.DebitCard@27c170f0, record_patterns.DebitCard@1f32e575
3: Two debit cards
```

1.6 Summary of Java Patterns

The syntax of type patterns and record patterns is summarized in **Figure 1.4**, together with their usage in the `instanceof` pattern match operator and the switch labels of the `switch` construct.

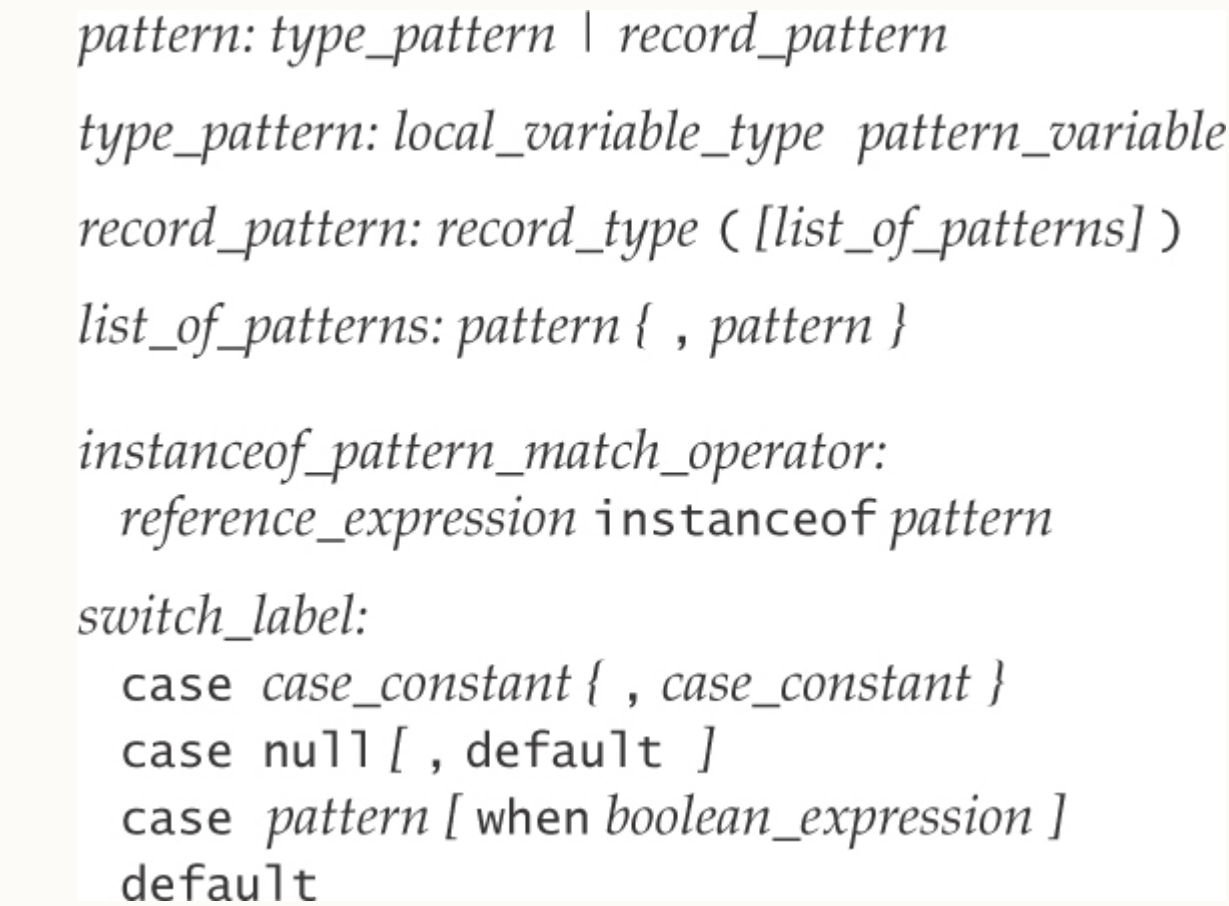


Figure 1.4 *Summary of Patterns*

1.7 Using Qualified Name for Enum Constants in the `switch` Construct

When the type of the selector expression is an enum type, a `switch` statement specified with enum constants in its `case` labels need not be exhaustive, regardless of whether it uses the colon (`:`) or the arrow (`->`) notation. However, a `switch` expression with enum constants in its `case` labels must always be exhaustive. See also **Table 1.2**, **p. 12**.

If the type of the selector expression is an enum type, enum constants can be specified in the `case` labels without their *qualified name*—that is without the name of the enum type. In the `switch` statement and expression below at (1) and (2), respectively, the enum constants of the enum type `MachineState` are specified in the `case` labels without the qualified name of the enum type.

Click here to view code image

```
// Enum declaration:
enum MachineState { BUSY, IDLE, BLOCKED }
...
MachineState ms = MachineState.IDLE;

switch (ms) {                  // (1) Switch statement need not be exhaustive.
  case IDLE: System.out.println("Doing nothing."); break;
  case BUSY: System.out.println("Too busy.");
}

String str = switch (ms) {      // (2) Switch expression must be exhaustive.
  case IDLE    -> "Doing nothing.";
  case BUSY    -> "Too busy.";
  case BLOCKED -> "Not available.";
};
```

As the type of the selector expression in the `switch` construct can be of *any* reference type, it can be necessary to use the qualified name of the enum constants, otherwise the compiler will not be able to resolve the name as an enum constant. In the code below, the enum constants are specified with their qualified name—otherwise the code will not compile. The `default` label is necessary, covering all possible values of the selector type, in order to make the `switch` expression exhaustive.

Click here to view code image

```
Object obj = MachineState.IDLE;  // Supertype reference denotes subtype object.

String str1 = switch (obj) {
  case MachineState.IDLE    -> "Doing nothing.";
  case MachineState.BUSY    -> "Too busy.";
  case MachineState.BLOCKED -> "Not available.";
  default                   -> "No match found for selector object.";
};
```

In summary, when the type of the selector expression in the `switch` construct is of an enum type, the names of its enum constants in the `case` labels can be qualified *or* unqualified—but if the selector expression is of any other reference type, the names of enum constants in the `case` labels *must* be qualified.

1.8 Unexpected Failure in Pattern Matching

Even though the `switch` construct may be deemed exhaustive at compile time, an unexpected failure can occur in pattern matching at runtime when the value does not match any of the specified patterns. In such cases, the unchecked `java.lang.MatchException` is thrown to indicate such failure.

What are known as *separate compilation anomalies* can result in pattern matching failure, where the type hierarchy that the patterns reference at runtime is different from what it was at compile time. For example, this happens if the set of enum constants of an enum type has changed at runtime from what it was at compile time, or if the set of permitted subtypes of a sealed type at runtime has changed from what it was at compile time.

We mention a couple of other cases where such pattern matching failure can occur. For example, if an accessor method throws an exception when extracting a component value during matching of a record pattern, pattern matching completes abruptly and will throw a `MatchException`.

Click here to view code image

```
// Record declaration:
record IllDefinedRecord(int value) {
  public int value() {
    return value / 0;              // Will throw an ArithmeticException.
  }
}
...
switch(new IllDefinedRecord(100)) {   // (1) MatchException!
  case IllDefinedRecord(var v) -> System.out.println(v);
}
```

Execution of the `switch` statement above at (1) will result in a `MatchException`:

Click here to view code image

```
Exception in thread "main" java.lang.MatchException: java.lang.ArithmeticExcep-
tion: / by zero
...

Caused by: java.lang.ArithmeticException: / by zero
...
```

Matching a nested record with `null` component values against a nested record patterns will result in runtime failure, as in the `switch` statement below at (2), where passing a `null` value instead of a `PhoneNumber` is the cause of the failure, and not that a `null` value is passed to the pattern variable `name`.

Click here to view code image

```
// Record declarations:
record PhoneNumber(int countryCode, int areaCode, int localNumber) { }
record Contact(String name, PhoneNumber phoneNumber) { } // Nested record
...
switch(new Contact(null, null)) {    // (2) MatchException!
  case Contact(var name, PhoneNumber(var cc, var ac, var ln))
    -> System.out.println(name);
}
```



Review Questions

1.1 Given the following code:

```
enum Condition {
  NEW, USED;
}
```

and

Click here to view code image

```
public String testCondition(Condition c) {
  String s = null;
  switch (c) {
    case NEW:
```

```
      s = "it's new";
      break;
    case USED:
      s = "it's used";
  }
  return s;
}
```

Which implementations of the `testCondition()` method are equivalent to the given method?

Select the two correct answers.

1. **Click here to view code image**

```
public String testCondition(Condition c) {
  String s = "it's used";
  switch (c) {
    case NEW ->
      s = "it's new";
  }
  return s;
}
```

2. **Click here to view code image**

```
public String testCondition(Condition c) {
  String s = "";
  switch (c) {
    case NEW ->
      s = "it's new";
      break;
    case USED ->
      s = "it's used";
      break;
  }
  return s;
}
```

3. **Click here to view code image**

```
public String testCondition(Condition c) {
 String s = "";
 switch (c) {
  case NEW -> {
   s = "it's new";
   break;
  }
  case USED -> {
   s = "it's used";
   break;
  }
 }
 return s;
}
```

4. **Click here to view code image**

```
public String testCondition(Condition c) {
 return  switch (c) {
 case NEW ->
  "it's new";
 case USED ->
  "it's used";
 };
}
```

5. **Click here to view code image**

```
public String testCondition(Condition c) {
 String s = "it's used";
 s = switch (c) {
  case NEW -> "it's new";
 };
 return s;
}
```

6. **Click here to view code image**

```
  public String testCondition(Condition c) {
    return switch (c) {
      case NEW -> "it's new";
      default:
        yield "it's used";
      }
    }
```

1.2 Given the following code:

Click here to view code image

```
public class Animal {
  public String toString() {
    return "animal";
  }
}

public class Dog extends Animal {
  public String toString() {
    return "dog";
  }
}

public class Cat extends Animal {
  public String toString() {
    return "cat";
  }
}

public class Test {
  static Animal x = new Dog();
  static Animal y = new Cat();
  public static void main(String[] args) {
    if (x instanceof Dog y) {
      System.out.println(y);
    } else {
      System.out.println(x);
    }
    System.out.println(y);
  }
}
```

What is the output from the program?

1. dog
   cat

2. animal
   dog

3. animal
   cat

4. dog
   animal
   dog

5. dog
   animal
   cat

6. Exception is thrown at runtime.
7. Compilation fails.

1.3 Given the following code:

Click here to view code image

```
public record Placeholder<T>(T value) { }
```

and

Click here to view code image

```
public class Test {
  public static void main(String[] args) {
    Placeholder<String> p = new Placeholder<>("acme");
    // (1) Insert code here.
```

```
  }
}
```

Which `if` statement is invalid when inserted into the `main()` method at (1)?

1. Click here to view code image

```
if (p instanceof Placeholder<?>(String value))
  System.out.println(value);
```

2. Click here to view code image

```
if (p instanceof Placeholder<>(String value))
  System.out.println(value);
```

3. Click here to view code image

```
if (p instanceof Placeholder<String>(String value))
  System.out.println(value);
```

4. Click here to view code image

```
if (p instanceof Placeholder(String value))
  System.out.println(value);
```

1.4 Given the following code:

Click here to view code image

```
public sealed interface Animal permits Dog, Cat { }

public record Dog(String name) implements Animal { }

public record Cat(String name) implements Animal { }

public class Test {
  public static void main(String[] args) {
    System.out.println(testAnimal(null));
```

```
  }
  public static String testAnimal(Animal animal) {
    return switch (animal) {
    // (1) Insert code here.
    };
  }
}
```

Which options will cause the program to compile and execute normally when inserted into the `switch` expression at (1)?

Select the two correct answers.

1. **Click here to view code image**

```
case null -> "none";
case Dog(String name) -> "Dog " + name;
case Cat(String name) -> "Cat " + name;
```

2. **Click here to view code image**

```
case null -> "none";
case Cat(String name) -> "Cat " + name;
case Animal animal -> "animal";
```

3. **Click here to view code image**

```
case Dog(String name) -> "Dog " + name;
case Cat(String name) -> "Cat " + name;
default -> "animal";
```

4. **Click here to view code image**

```
case null -> "none";
case Dog(String name) -> "Dog " + name;
case Animal some -> "animal";
```

5. **Click here to view code image**

```
case Cat(String name) -> "Cat " + name;
case null, default -> animal.toString();
```

1.5 Method `test()` should accept any `Object` as an argument. If this argument is a `null` or is not a `String`, it should return `3`. If the argument is a mixed-case `String`, it should return `2`. If the argument is a lower-case `String`, it should return `0`. If the argument is an upper-case `String`, it should return `1`.

Which implementations of method `test()` correctly implements this scenario? Select the three correct answers.

1. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case null -> 3;
    case Object o -> 3;
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
  };
}
```

2. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case null -> 3;
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    default -> 3;
  };
}
```

3. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
```

```
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    case null -> 3;
    default -> 3;
  };
}
```

4. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case String s -> 2;
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case Object o -> 3;
    case null -> 3;
  };
}
```

5. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    case Object o -> 3;
    case null -> 3;
  };
}
```

1.6 Method `test()` should accept any `Object` as an argument. If this argument is a `null` or is not a `String`, it should return `3`. If the argument is a mixed-case `String`, it should return `2`. If the argument is a lower-case `String`, it should return `0`. If the argument is an upper-case `String`, it should return `1`.

Which implementation of method `test()` correctly implements this scenario?

1. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case null || Object o -> 3;
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
  };
}
```

2. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    case null, default -> 3;
  };
}
```

3. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    case default, null -> 3;
  };
}
```

4. **Click here to view code image**

```
public int test(Object obj) {
  return switch(obj) {
    default -> 3;
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
```

```
    case null -> 3;
  };
}
```

5. Click here to view code image

```
public int test(Object obj) {
  return switch(obj) {
    case String s when s.toLowerCase().equals(s) -> 0;
    case String s when s.toUpperCase().equals(s) -> 1;
    case String s -> 2;
    case Object o -> 3;
    case null, default -> 3;
  };
}
```

1.7 Which classes when executed print `Cake 1.99 2`?

Select the two correct answers.

1. Click here to view code image

```
public class Q7A {
  static String name = "Cake";
  static double price = 1.99;
  static int quantity = 2;

  public static void main(String[] args) {
    record Order(Product product, int quantity) {
      record Product(String name, double price) {}
    }
    Order order = new Order(new Order.Product(name, price), quantity);
    if (order instanceof Order anOrder) {
      System.out.println(anOrder.product.name + " " +
                         anOrder.product.price + " " +
                         anOrder.quantity);
    }
  }
}
```

2. Click here to view code image

```
public class Q7B {
  public static void main(String[] args) {
  String name = "Cake";
  double price = 1.99;

    int quantity = 2;
    record Order(Product product, int quantity) {
      record Product(String name, double price) {}
    }
    Order order = new Order(new Order.Product(name, price), quantity);
    if (order instanceof Order(Order.Product(var name, var price),
                               var quantity)) {
      System.out.println(name + " " + price + " " + quantity);
    }
  }
}
```

3. Click here to view code image

```
public class Q7C {
  public static void main(String[] args) {
    record Order(Product product, int quantity) {}
    record Product(String name, double price) {}
    String name = "Cake";
    double price = 1.99;
    int quantity = 2;
    Order order = new Order(new Product(name, price), quantity);
    if (order instanceof Order(Product(var name, var price),var quantity)) {
      System.out.println(name + " " + price + " " + quantity);
    }
  }
}
```

4. Click here to view code image

```
public class Q7D {
  record Order(Product product, int quantity) {}
  record Product(String name, double price) {}
```

```
  static String name = "Cake";
  static double price = 1.99;
  static int quantity = 2;

  public static void main(String[] args) {
    Order order = new Order(new Product(name, price), quantity);
    if (order instanceof Order(Product(var name, var price),var quantity)) {
      System.out.println(name + " " + price + " " + quantity);
    }

  }
}
```

Previous chapter

< Preface to OCP Java SE 21 Developer (Exam 1Z0-830)

Next chapter

2. Sequenced Collections and Maps >